

AI-Powered Incident & Management Support System

Project System Design & Development Document

Focus: Backend Engineering + AI Engineering

Primary Backend: Python + FastAPI

AI: LLM + RAG + LangChain + LangGraph

Development Plan: 10 Days / 20 Phases

Document Contents

- 1. Project Overview
- 2. Business Functional Specification (BFS)
- 3. Functional Requirements Specification (FRS)
- 4. Technology Stack
- 5. High-Level System Design
- 6. 10-Day / 20-Phase Development Plan
- 7. Final Project Outcome

1. Project Overview

Project Name: AI-Powered Incident & Management Support System

Objective: Build a backend platform that allows organizations to report, manage, track, analyze, and resolve technical incidents with AI assistance.

The system combines traditional incident management with LLM-based classification, semantic search, Retrieval-Augmented Generation (RAG), and AI-agent capabilities.

Core Incident Flow

User reports incident → FastAPI Backend → Incident stored in PostgreSQL → AI analyzes incident → Similar incidents / knowledge retrieved → AI recommendation → Engineer investigates → Resolution → AI-generated resolution summary.

High-Level Architecture

Client / Web / API → FastAPI API Layer → PostgreSQL + AI Service. The AI service connects to an LLM, RAG pipeline, embeddings, and Qdrant vector database.

2. Business Functional Specification (BFS)

2.1 Business Objective

Provide an intelligent platform to report, manage, prioritize, investigate, and resolve IT incidents, reducing manual investigation effort and improving operational visibility and resolution efficiency.

2.2 Business Users

Role	Responsibility
Admin	Manage users, teams, configuration, and system-level information.
Support Engineer	Investigate and resolve assigned incidents using system and AI assistance.
Reporter	Submit incidents, track progress, and provide additional information.
Manager	Monitor incidents, workload, escalation, and operational metrics.

2.3 Core Business Functions

BF-01 — Incident Reporting

Users can report technical incidents. Each incident receives a unique ID and is stored for tracking.

BF-02 — Incident Classification

Incidents are categorized by type, severity, and priority. AI can recommend these values.

BF-03 — Incident Assignment

Incidents are assigned to appropriate teams or engineers. Managers can reassign incidents.

BF-04 — Incident Tracking

Track incidents through Open → Triaged → In Progress → Resolved → Closed.

BF-05 — AI-Assisted Resolution

Analyze incidents, find similar historical incidents, search knowledge-base information, recommend troubleshooting steps, and generate resolution summaries.

BF-06 — Knowledge Management

Store troubleshooting guides, runbooks, FAQs, and previous resolutions for search and AI retrieval.

BF-07 — Escalation

High-priority incidents can be escalated and relevant teams/users can be notified.

BF-08 — Reporting & Analytics

Monitor incident volume, open/resolved incidents, priority distribution, resolution time, team workload, and recurring incidents.

2.4 Business Benefits

Faster incident resolution → reduced manual investigation → better reuse of organizational knowledge → improved support productivity → better incident visibility.

2.5 Business KPIs

- MTTR — Mean Time to Resolution
- Incident resolution rate
- Average response time
- Number of recurring incidents
- AI recommendation acceptance rate
- Incidents resolved using knowledge-base recommendations

3. Functional Requirements Specification (FRS)

3.1 User Roles

Role	Functional Access
Admin	Manage users, teams, configuration, all incidents, analytics, and knowledge-base documents.
Support Engineer	View assigned incidents, update status, comment, resolve incidents, and use AI recommendations.
User / Reporter	Create incidents, view submitted incidents, comment, and view resolution details.

FR-01 — Authentication

- User registration/login
- JWT authentication
- Role-based access control
- Logout/session revocation
- Protected APIs

FR-02 — User Management

- Create/update users
- Deactivate users
- Assign users to teams
- Assign roles

FR-03 — Incident Management

- Create/view/search/update incidents
- Add comments
- Attach relevant information/files
- Track reporter, assignee, team, dates, category, priority, severity

FR-04 — Incident Lifecycle

- OPEN → TRIAGED → IN_PROGRESS → RESOLVED → CLOSED
- Only authorized users can change states

FR-05 — AI Incident Classification

- Recommend category, severity, priority, component, and team
- Users can review or override AI recommendations

FR-06 — AI Incident Summarization

- Problem summary
- Investigation steps
- Actions taken
- Current status
- Resolution summary

FR-07 — Similar Incident Detection

- Semantic search over historical incidents
- Display relevant previous incidents

FR-08 — Knowledge Base

- Create/update/delete articles
- Upload knowledge documents
- Search articles
- Support runbooks, guides, FAQs, and resolutions

FR-09 — RAG-Based AI Support

- Embed user query
- Retrieve relevant documents
- Search similar incidents
- Provide context to LLM
- Generate contextual response

FR-10 — AI Root-Cause Analysis

- Possible causes
- Supporting evidence
- Historical incidents
- Recommended investigation
- Suggested resolution
- Separate hypotheses from confirmed causes

FR-11 — AI Support Agent

- Analyze incidents
- Search knowledge
- Search historical incidents
- Retrieve incident data
- Recommend actions
- Generate responses

FR-12 — Notifications

- Assignment notification
- Priority change
- Status change
- Escalation
- Resolution notification

FR-13 — Audit Logging

- User
- Action
- Incident
- Timestamp
- Previous value
- New value

FR-14 — Search & Filtering

- Incident ID
- Status
- Priority
- Severity
- Category
- Team
- Engineer
- Date range

FR-15 — Analytics

- Total incidents

- Open/resolved
- Average resolution time
- Priority/category distribution
- Team workload

3.2 Non-Functional Requirements

Area	Requirements
Performance	Fast normal API responses; AI processing should not block normal API operations.
Security	JWT, RBAC, password hashing, validation, secure endpoints, audit logging.
Scalability	API, AI services, workers, database, and vector database should be independently scalable.
Reliability	Error handling, transactions, retries for AI/background operations, logging and monitoring.
Maintainability	Modular FastAPI architecture, service/repository separation, automated tests, Docker.

3.3 MVP Scope

Phase 1: Authentication, users, teams, incident CRUD, lifecycle, comments, PostgreSQL.

Phase 2: AI classification, priority recommendation, summarization.

Phase 3: Knowledge base, embeddings, Qdrant, RAG, similar-incident search.

Phase 4: LangGraph agent, tool calling, root-cause assistance, notifications, analytics.

Phase 5: Docker, CI/CD, AWS, monitoring, production deployment.

4. Technology Stack

Layer	Technology
Language	Python
Backend	FastAPI
Database	PostgreSQL
ORM	SQLAlchemy
Authentication	JWT + OAuth2
Validation	Pydantic
LLM	Ollama / OpenAI-compatible API
AI Framework	LangChain
Agent Framework	LangGraph
Embeddings	HuggingFace
Vector Database	Qdrant
Cache / Queue	Redis
Background Jobs	Celery
Testing	Pytest
API Testing	Postman

Layer	Technology
Containerization	Docker
CI/CD	GitHub Actions
Cloud	AWS
Monitoring	Prometheus + Grafana
Version Control	Git + GitHub

Core Learning Stack

Python → FastAPI → PostgreSQL + SQLAlchemy → JWT Authentication → LLM → Embeddings + Qdrant → RAG → LangChain → LangGraph.

5. High-Level System Design

5.1 Main Architecture

Client / Web / API → FastAPI API Gateway → PostgreSQL and AI Service → LLM + RAG + Agent + Embeddings → Qdrant.

5.2 Main Modules

- Authentication
- Incident Management
- AI Incident Analysis
- Knowledge Base / RAG
- Similar Incident Search
- AI Support Agent
- Notifications & Escalation
- Analytics
- Audit Logging

5.3 Incident Lifecycle

Create Incident → Store in PostgreSQL → AI Classification → Priority/Severity → Similar Incident Search → Knowledge Retrieval → AI Analysis → Team Assignment → Engineer Investigation → Resolution → AI Resolution Summary → Close Incident.

5.4 AI Architecture

Incident → AI Classifier → Similar Incidents + Knowledge Base → RAG → LLM/Agent → Diagnosis + Recommendation.

5.5 Core Database Entities

Users, Teams, Incidents, IncidentComments, KnowledgeArticles, Attachments, AIAalysis, AuditLogs, Notifications.

6. 10-Day / 20-Phase Development Plan

Development sprint: 2 phases per day. The plan is aggressive and should be treated as an MVP implementation schedule rather than a claim that every technology can be mastered in ten days.

Day	Phase	Development Work
Day 1	Phase 1 — Project Setup	FastAPI structure, .env, configuration, logging, requirements, Docker basics.

Day	Phase	Development Work
Day 1	Phase 2 — Database Setup	PostgreSQL, SQLAlchemy, Alembic, database connection, base models.
Day 2	Phase 3 — Authentication	User model, password hashing, JWT login/register, token validation.
Day 2	Phase 4 — RBAC & Users	Admin/Engineer/Reporter/Manager roles, permissions, user/team management.
Day 3	Phase 5 — Incident Module	Incident model, schemas, CRUD APIs, validation.
Day 3	Phase 6 — Incident Lifecycle	Status transitions, priority, severity, assignment, comments, audit records.
Day 4	Phase 7 — Search & Filtering	Search, pagination, filtering by status/priority/category/team/date.
Day 4	Phase 8 — Notification System	Notification model, assignment/status notifications, Redis/background processing.
Day 5	Phase 9 — LLM Integration	LLM client, prompts, structured outputs, AI service abstraction.
Day 5	Phase 10 — AI Classification	Automatic category, severity, priority, component and team recommendation.
Day 6	Phase 11 — AI Summarization	Incident summaries, comment summaries, resolution summaries.
Day 6	Phase 12 — Knowledge Base	Knowledge CRUD, document upload, text extraction, document management.
Day 7	Phase 13 — Embedding Pipeline	Chunk documents, generate embeddings, store vectors in Qdrant.
Day 7	Phase 14 — RAG Pipeline	Query → embedding → similarity search → retrieved context → LLM response.
Day 8	Phase 15 — Similar Incident Detection	Incident embeddings and semantic search against historical incidents.
Day 8	Phase 16 — AI Resolution Assistant	Combine incident + similar incidents + RAG to produce troubleshooting recommendations.
Day 9	Phase 17 — LangGraph Agent	Agent workflow, state, tools, RAG/tool routing.
Day 9	Phase 18 — Human-in-the-Loop	AI recommendation → engineer review → approve/reject → action/update.
Day 10	Phase 19 — Testing & Production Hardening	Unit test, API/AI tests, error handling, security, logging, Docker.
Day 10	Phase 20 — Deployment & Documentation	Docker deployment, AWS, CI/CD, API documentation, architecture diagram, README.

6.1 Detailed Development Structure

Day 1 — Phase 1: Project Setup

- Create app/, api/, models/, schemas/, services/, repositories/, ai/, core/.
- Create FastAPI application, environment configuration, .env, logging, exception handling, CORS, Dockerfile.

Day 1 — Phase 2: Database Setup

- Set up PostgreSQL.
- Configure SQLAlchemy and Alembic.
- Create initial User, Team, and Incident models.

Day 2 — Phase 3: Authentication

- Implement POST /auth/register, POST /auth/login, GET /auth/me.
- Implement password hashing, JWT, OAuth2 password flow, access-token validation.

Day 2 — Phase 4: RBAC & Users

- Implement ADMIN, ENGINEER, REPORTER, MANAGER.
- Create permission dependencies and user/team management.

Day 3 — Phase 5: Incident Module

- Implement POST/GET/PATCH/DELETE /incidents.
- Add title, description, category, priority, severity, status, reporter, assignee, and team.

Day 3 — Phase 6: Incident Lifecycle

- Implement OPEN → TRIAGED → IN_PROGRESS → RESOLVED → CLOSED.
- Add comments, assignment, and audit logs.

Day 4 — Phase 7: Search & Filtering

- Implement status/priority/team/search filters.
- Add pagination, sorting, and date filtering.

Day 4 — Phase 8: Notifications

- Use FastAPI + Redis + background worker.
- Trigger assignment, priority, escalation, and resolution notifications.

Day 5 — Phase 9: LLM Integration

- Create app/ai/llm.py, prompts.py, and schemas.py.
- Keep LLM calls behind a service abstraction rather than directly inside routes.

Day 5 — Phase 10: AI Classification

- Input incident description.
- Return category, severity, priority, component, and recommended team as structured output.

Day 6 — Phase 11: AI Summarization

- Generate incident, investigation, and resolution summaries.

Day 6 — Phase 12: Knowledge Base

- Create KnowledgeArticle model and CRUD APIs.
- Support PDF/TXT/Markdown document ingestion.

Day 7 — Phase 13: Embedding Pipeline

- Extract text, chunk documents, generate embeddings, and store vectors/metadata in Qdrant.

Day 7 — Phase 14: RAG Pipeline

- User question → embedding → Qdrant top-K retrieval → context → LLM → answer.
- Expose POST /ai/ask.

Day 8 — Phase 15: Similar Incidents

- Embed incidents and search Qdrant for semantically similar historical incidents.

Day 8 — Phase 16: Resolution Assistant

- Combine current incident, similar incidents, and knowledge-base context to generate evidence-based recommendations.

Day 9 — Phase 17: LangGraph Agent

- Build stateful workflow with analysis, retrieval, similar-incident search, and response nodes.
- Add tools such as search_incidents(), search_knowledge(), get_incident().

Day 9 — Phase 18: Human-in-the-Loop

- Require engineer review before sensitive AI-recommended actions.
- Support approve/reject workflow.

Day 10 — Phase 19: Testing & Hardening

- Test authentication, CRUD, RBAC, classification, RAG, similar incidents, and agent workflows.
- Add global error handling, request logging, security headers, production configuration, and rate limiting where appropriate.

Day 10 — Phase 20: Deployment & Documentation

- Containerize, configure CI/CD, deploy to AWS, document APIs, architecture, setup, and demo flow.

7. Final Project Outcome

The completed system should demonstrate five major engineering areas:

BACKEND: FastAPI + PostgreSQL + JWT + RBAC

AI: LLM + structured outputs

RAG: embeddings + Qdrant + retrieval

AGENTS: LangGraph + tools + human approval

PRODUCTION: Docker + testing + CI/CD + AWS

Final End-to-End Flow

Create Incident → AI Classification → Similar Incident Search → Knowledge Retrieval → RAG → AI Resolution Recommendation → Engineer Review → Incident Resolution → AI Resolution Summary → Close Incident.